

Операционные системы, практика

Введение

Юрий Литвинов
y.litvinov@spbu.ru

05.09.2026

Формальные вопросы

- ▶ Занятия по субботам раз в две недели в 3389
 - ▶ Две пары практики сразу, чтобы успеть сделать что-то содержательное
 - ▶ Будем работать прежде всего на парах, домашек не будет (если только доделать что-то)
 - ▶ Работа в основном командная
- ▶ Берите с собой ноуты с qemu
- ▶ Курс на HwProj: <https://hwproj.ru/courses/50095>
- ▶ Мои контакты:
 - ▶ Почта: y.litvinov@spbu.ru
 - ▶ MAX: <https://max.ru/u/f9LHodD0cOKMROGyAurSdR8veLXtPdnffzAiuNC71QqGMlcHkAvdwAzZeAl>

Критерии оценивания

- ▶ Всего примерно 7 заданий, каждое оценивается из 10 баллов
 - ▶ Можно один раз пересдать, чтобы улучшить балл
 - ▶ -3 невозможных за пропуск дедлайна
 - ▶ Делаем в основном на паре, но если пропустили, можно досдать
- ▶ Баллы за домашние работы переводятся в шкалу 0-40
- ▶ Будут две «контрольные» — индивидуальная устная защита выполненных заданий
- ▶ Могут быть летучки с теоретическими вопросами на 5 минут, на любой паре
 - ▶ так что даже в командах надо понимать *всё*, что делается
- ▶ В конце — теоретический зачёт по шкале 0-60

Что будет в курсе

- ▶ Базовая работа в xv6 — установка/запуск, написание простых приложений с системными вызовами, добавим свой системный вызов
- ▶ Работа с таблицей процессов
- ▶ Многопоточность и синхронизация, мьютексы
- ▶ Трассировка событий ядра, dmesg
- ▶ Межпроцессное взаимодействие
- ▶ Написание драйверов
- ▶ Файловые системы — чтение образа ext2

Практики могут немного обгонять лекции!

Краткий обзор xv6

- ▶ Учебная операционная система для курса MIT по ОС
 - ▶ <https://pdos.csail.mit.edu/6.1810/2026/index.html>
- ▶ POSIX-like
 - ▶ Не поддерживает много чего из POSIX, зато простая
 - ▶ «Почти Linux»
- ▶ Важные отличия от обычных ОС:
 - ▶ Один поток на процесс
 - ▶ Очень простой менеджер памяти и планировщик
 - ▶ Собираем образ прямо с прикладной программой (как во встроенных системах)
 - ▶ Заточена под 64-битный RISC-V
- ▶ Репозиторий: <https://github.com/mit-pdos/xv6-riscv>
- ▶ Инструкция по сборке и установке:
<https://pdos.csail.mit.edu/6.1810/2026/tools.html>

Структура исходников

- ▶ kernel — монолитное ядро системы
 - ▶ entry.S — начальный загрузчик (на ассемблере RISC-V)
 - ▶ main.c — собственно инициализация системы
 - ▶ proc.c — процессы и планировщик
 - ▶ kernelfvec.S / trampoline.S — обработчики прерываний в режиме ядра / пользовательском режиме
 - ▶ syscall.c — диспетчер системных вызовов
 - ▶ console.c — код для работы с консолью
 - ▶ pipe.c — код работы с *пайпами*
 - ▶ ...
- ▶ user — программы пространства пользователя
 - ▶ sh.c — командная оболочка
 - ▶ ...
- ▶ mkfs — утилита, собирающая загрузочный образ

Системные вызовы

- ▶ `int fork()` — создаёт новый процесс, копию текущего
- ▶ `int exit(int status)` — заканчивает текущий процесс
- ▶ `int wait(int *status)` — блокирует текущий процесс, пока не закончится дочерний
- ▶ `int pause(int n)` — блокирует текущий процесс на `n` тиков
- ▶ `int getpid()` — возвращает ID текущего процесса
- ▶ `int kill(int pid)` — завершить указанный процесс
- ▶ `int dup(int fd)` — создать новый файловый дескриптор для `fd`
- ▶ `int pipe(int p[])` — создать пайп
- ▶ ...

Полезные функции ядра

- ▶ `argint` — TODO

Ввод-вывод

- ▶ С каждым процессом ассоциирована таблица дескрипторов
- ▶ Дескриптор с индексом 0 — stdin, 1 — stdout, 2 — stderr
- ▶ Их можно подменять: `close(0); open("input.txt", O_RDONLY);`
 - ▶ `open` гарантированно выберет минимальный свободный индекс
 - ▶ в процессе всегда один поток, так что никто не «украдёт» ячейку под дескриптор

▶ Пайп:

```
int p[2];
char *argv[2];
argv[0] = "wc";
argv[1] = 0;
pipe(p);
if (fork() == 0) {
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    exec("/bin/wc", argv);
} else {
    close(p[0]);
    write(p[1], "hello world\n", 12);
    close(p[1]);
}
```

Задача на пару (1)

На 3 балла

- ▶ Задача 1. Написать программу, выполняющую следующие действия в xv6.
 - ▶ Создается дочерний процесс, который приостанавливается (pause) 5-15 секунд и завершается с кодом возврата 1 (exit).
 - ▶ Родительский процесс выводит свой идентификатор (getpid) и идентификатор дочернего процесса, затем
 - ▶ (a) просто ожидает завершения дочернего процесса (wait), после чего выводит идентификатор завершенного процесса и его код возврата, затем сам завершается (exit)
 - ▶ (b) сначала посылает дочернему процессу сигнал kill, после чего ожидает его завершения, выводит идентификатор завершенного процесса и код возврата, затем сам завершается
- ▶ В варианте (a) протестировать запуск приложения в виде фонового процесса оболочки и отправку сигнала дочернему процессу с помощью команды kill, не дожидаясь его завершения по тайм-ауту.

Задача на пару (2)

На 3 балла

Задача 2. Добавить в xv6 системный вызов, вычисляющий сумму двух целых чисел (своих аргументов).

- ▶ Для доступа к аргументам системного вызова использовать соответствующие функции (argint)
- ▶ Вернуть сумму чисел как результат системного вызова
- ▶ Добавить отладочный вывод
- ▶ Протестировать работу с помощью утилиты пользовательского пространства

Задача на пару (3)

На 4 балла

Задача 3*. Написать программу, которая создает дочерний процесс и передает ему через заранее созданный анонимный канал (pipe) все аргументы командной строки с помощью write, завершая их символом перевода строки '\n', закрывает канал по завершении записи. Дочерний процесс замещает стандартный ввод (0) выходным концом канала (pipefd[0]) с помощью dup: Программа wc должна вывести подсчет строк (количества аргументов командной строки), слов и байтов в них. Родительский процесс ожидает завершения дочернего процесса (wait) и завершается сам (exit).

Задача на пару (4)

На 3 балла, бонусная

Задача 4. (бонусная). Реализовать в "настоящей" ОС (POSIX) программу, которая создает дочерний процесс и передает ему через заранее созданный анонимный канал (pipe) все аргументы командной строки с помощью write, завершая их символом перевода строки '\n', закрывает канал по завершении записи. Дочерний процесс читает данные из канала с помощью read и выводит их на стандартный вывод.

Обратите внимание! (1)

- ▶ Вызов pause в xv6 считает "ticks" (примерно 1/10 секунды, вычисляется как определенное число инструкций процессора), подсчета "настоящего" времени нет.
- ▶ Хотя ОС при завершении процесса должна закрыть все открытые файловые дескрипторы, рекомендуется закрывать используемые концы канала и иные файлы после работы.
- ▶ Закрытие неиспользуемых концов канала обязательно, т.к. иначе может возникнуть взаимоблокировка.
- ▶ Следует проверять на ошибки все используемые функции и системные вызовы, и не допускать неопределенного поведения, в т.ч. при создании процессов, каналов, вызова `exec`, чтения и записи, в т.ч. вызова `close` после записи, т.к. нарушения в работе (например, аппаратные для файлового вывода, или программный обрыв канала `pipe` по каким-либо причинам могут привести к тому, что не все данные будут записаны/переданы.

Обратите внимание! (2)

- ▶ Вызов `exes` может завершиться с ошибкой (например, отсутствие указанной программы для запуска). В этом случае исполняется следующий за ним код процесса. Если `exes` выполняется успешно, выполняется код вызванной программы, возврата в исходный процесс не происходит.
- ▶ Оболочка `xv6` не поддерживает экранирование пробелов и других символов (кавычки и т.п.), поэтому `ws` будет возвращать равное количество слов и строк. Но в общем случае можно вызывать данную программу из другой с помощью `exes`. В этом случае возможны аргументы с пробелами, а также аргументы, содержащие символ перевода строки. Поэтому, вообще говоря, искусственное добавление `'\n'` к каждому аргументу в родительском процессе не гарантирует, что `ws` посчитает число аргументов исходного процесса, тем более в реальной системе.

Обратите внимание! (3)

- ▶ Оболочка xv6 по каким-то причинам ограничивает количество аргументов приложения сильнее, чем ядро (10 против 32). Это можно пропатчить.
- ▶ Формально, успехом write считается любое положительное число выведенных байтов. То есть write может вывести меньше данных, чем указано, и это не будет ошибкой - следует корректно выводить содержимое буфера, пока он не будет выведен полностью или пока не будет получена ошибка (отрицательное значение). Аналогичная ситуация с read: реальное количество прочитанных байтов может быть меньше размера буфера, чтение нужно проводить до достижения конца файла (0) или до ошибки (отрицательное значение).